CHMLTECH ONLINE JOURNAL · VOLUME 105, ISSUE 10 · RESEARCH ARTICLE

Evaluating AI Agents: Why Public Benchmarks Mislead and How to Build Your Own Evaluation

Quak Foo Lee

Chief Technology Officer, Chmltech Ltd.

Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 13, 2026 14 min read

ABSTRACT

Benchmark scores shape which AI agents businesses buy and deploy, yet recent work shows that they can be badly wrong. In April 2026 researchers at the University of California, Berkeley reported an automated agent that reached near-perfect scores on eight widely cited agent benchmarks without solving a single task, by exploiting weaknesses in how scores were computed. An earlier audit of ten agentic benchmarks found flaws that could under- or overestimate performance by up to 100 percent in relative terms. This paper explains what public agent benchmarks measure and why their scores mislead: weak graders, tasks unlike a business's own, scaffold effects, single-run scoring that hides inconsistency, missing cost data and biased automated judges. It then sets out how a business can build its own evaluation: a task set drawn from real cases, an outcome definition for each task, a grader isolated from the agent, repeated runs to measure reliability with the pass^k metric, confidence intervals sized to the decision, and cost and safety cases alongside accuracy. Worked calculations show that around 400 tasks are needed to estimate a 90 percent success rate to within three percentage points. The paper closes with how evaluation results should gate deployment, autonomy and model changes.

Keywords: AI agents; evaluation; benchmarks; pass^k; LLM-as-a-judge; reliability; test sets; regression testing

1. Introduction

When a business chooses a model or an agent product, the first numbers it sees are benchmark scores. Vendors publish them, analysts compare them and leaderboards rank them. The implicit promise is that a higher score means a more capable system.

In April 2026 a team at the University of California, Berkeley put that promise to an adversarial test. They built an automated agent that analyzes how a benchmark computes its score and looks for ways to inflate it, and pointed it at eight of the most widely cited agent benchmarks. Without solving a single task, and in most cases without calling a language model at all, it reached near-perfect scores on seven of them and 73 percent on the eighth [1]. On SWE-bench Verified, a ten-line configuration file that forced the test framework to report every test as passed was enough. On Terminal-Bench, wrapping a system utility so that it returned the expected output produced a perfect score on all 89 tasks [1]. The researchers did not claim that published results were fraudulent; their point was that the benchmarks could not distinguish a capable agent from one that manipulated the grader.

A year earlier, Zhu and colleagues had audited ten popular agentic benchmarks against a checklist of good practice and found that seven had flaws in how outcomes were judged and seven had flaws in task setup, with errors that could under- or overestimate performance by up to 100 percent in relative terms [2].

This paper draws the practical consequences for a business. Section 2 describes what public agent benchmarks measure. Section 3 explains why their scores mislead. Section 4 sets out how to build an evaluation from a business's own work, with the statistics needed to size it. Section 5 covers automated judges, and Section 6 explains how evaluation results should drive decisions about deployment, autonomy and model changes.

2. What public agent benchmarks measure

Agent benchmarks differ from earlier language-model tests in that the model acts in an environment over many steps, and success is judged by the outcome rather than by a single answer. Table 1 summarizes five influential examples.

Table 1. Five widely used agent benchmarks.

Benchmark	Environment	Tasks	How success is judged
SWE-bench	Real GitHub repositories	2,294 issues from 12 Python projects (500 in the human-validated Verified subset)	The agent's code change makes the project's tests pass
WebArena	Self-hosted websites: shopping, forum, code hosting, content management	812 tasks	Checks on the final page state or the returned answer
OSWorld	Real desktop operating systems and applications	369 tasks	Scripts that inspect the final state of files and applications
GAIA	Open web and tools	466 questions	Exact match against a short reference answer
τ-bench	Simulated customer service (retail, airline) with a simulated user and policies	Retail and airline task sets	Final database state compared with the goal state; reliability over repeated runs

Sources: [3], [4], [5], [6], [7].

These benchmarks were major advances. SWE-bench, for example, replaced artificial coding puzzles with real issues from real projects [3], and τ-bench introduced the pass^k metric, which measures whether an agent succeeds on all of *k* repeated attempts at the same task [7]. Their limitations come from the difficulty of judging open-ended outcomes automatically and at scale.

3. Why benchmark scores mislead

3.1 The grader can be fooled

Most benchmarks run the agent and the grader in the same environment, so the agent can, in principle, alter what the grader sees. The Berkeley team found that the most common weakness was exactly this lack of isolation between the agent and the evaluator, followed by

answers shipped with the test data, graders that executed agent output and automated judges whose prompts the agent could influence [1]. Zhu and colleagues found subtler problems, such as tests too weak to detect a wrong solution and a benchmark that counted empty responses as successes [2]. A business does not need to worry that vendors are gaming leaderboards to learn the lesson: if the grader can be fooled, a high score may not mean what it appears to.

3.2 The tasks are not your tasks

Even a perfectly graded benchmark measures performance on its own tasks. A high score on web navigation in a simulated shop says little about an agent's ability to code supplier invoices to a particular chart of accounts, follow a particular returns policy or read a particular plant's maintenance history. Liang and colleagues argued for evaluating language models across many scenarios and metrics precisely because performance does not transfer reliably from one scenario to another [8].

3.3 The scaffold matters as much as the model

An agent is a model plus a scaffold: the prompts, tools, memory, retries and control logic around it. Kapoor and colleagues showed that scaffolds can change results substantially and that simple, cheap designs often match complex ones, so a benchmark result describes a model-and-scaffold combination, not the model alone [9]. A business that deploys the same model in a different scaffold should not expect the published score.

3.4 One run hides inconsistency

Most leaderboards report the share of tasks solved on a single attempt. Yao and colleagues found that agents which solved a task once often failed on repeated attempts: in the retail domain, a leading model succeeded on all eight attempts at the same task in fewer than 25 percent of tasks [7]. For a business process that repeats thousands of times, consistency matters more than peak performance.

3.5 Cost is missing

Benchmarks usually report accuracy without the cost of achieving it. Kapoor and colleagues argued that accuracy and cost must be reported together, since an agent that calls a model many times can buy accuracy at a price that would never be acceptable in production [9]. A companion paper shows how quickly cost per task grows with the number of agent steps [10].

3.6 Automated judges have biases

Where outcomes are free text, benchmarks often use a language model as the judge. Zheng and colleagues found that strong models used as judges agreed with human preferences at a rate similar to the agreement between humans, but also showed position bias (favoring the first of two answers), verbosity bias (favoring longer answers) and self-enhancement bias (favoring answers in their own style) [11]. The Berkeley team also showed that an agent can embed instructions in its own output that steer an unprotected judge [1].

3.7 Public tasks leak

A benchmark that is published becomes, over time, part of the material on which future models are trained and against which developers tune their systems. Questions and reference answers posted online can be memorized, and the Berkeley team noted that the validation answers for one benchmark were publicly hosted [1]. Benchmark maintainers counter this with hidden test sets and fresh tasks, but a business cannot easily tell how much of a published score reflects capability and how much familiarity. A private evaluation built from the business's own records avoids the problem, provided those records are not themselves published.

> Use public benchmarks to shortlist models and products, not to decide what to deploy. The deployment decision needs an evaluation built from the business's own tasks, graded in a way the agent cannot influence.

4. Building your own evaluation

4.1 Start from real cases

The foundation of an evaluation is a task set drawn from the business's own work. For an accounts payable agent, that means past invoices; for a customer service agent, past tickets; for a maintenance agent, past fault reports with their eventual diagnosis. Three principles apply.

1. **Sample, then stratify.** Draw a random sample from recent history so the set reflects the real mix of work, then add hard and unusual cases deliberately: the invoice with a missing purchase order, the ticket in which the customer contradicts themselves, the fault that turned out to have two causes.

2. **Define the correct outcome before running the agent.** For structured work this is the set of fields that must be right, such as supplier, amount, account code and tax treatment. For unstructured work it is a rubric with specific criteria.
3. **Keep a held-out set.** If the team tunes instructions against the whole set, scores will rise without the agent improving. Hold back a portion that is used only for the final check before release.

Behavioral test suites from software engineering are a useful complement. Ribeiro and colleagues showed that systematic tests of specific capabilities and invariances, for example that changing a supplier's name format should not change the account code, find failures that aggregate accuracy hides [12].

4.2 Isolate the grader

The lesson of Section 3.1 is architectural. The agent should run in an environment it controls; the grader should run afterwards, in a separate environment, reading only the outputs the agent was supposed to produce: the record written to the test system, the email it drafted, the answer it returned. Reference answers should never be visible to the agent, and the grader should not execute anything the agent wrote. These are the same controls a business applies to production systems, and they are discussed further in a companion paper on agent security [13].

4.3 Grade the outcome and the path

Outcome checks answer whether the result was right. Path checks answer whether the agent got there acceptably: whether it called only permitted tools, stayed within value limits, asked for approval where required and did not expose data it should not have. An agent that reaches the right answer by violating a policy has failed. Logging each run's full trajectory of tool calls makes path checks possible and also helps diagnose failures.

4.4 Measure reliability

Run each task several times. Yao and colleagues define pass^k as the probability that all k independent attempts at a task succeed, estimated from n trials per task with c successes as

$$\text{pass\^{}k} = \mathrm{E}_{\text{tasks}}\left[\, C(c, k) \,/\, C(n, k) \,\right] \tag{1}$$

where $C(\cdot,\cdot)$ is the binomial coefficient and the expectation is taken over tasks [7]. If attempts were independent with the same success probability p on every task, pass^k would simply be p^k. Figure 1 shows how quickly that falls: an agent with a 90 percent per-attempt success rate completes all of eight attempts only 43 percent of the time.

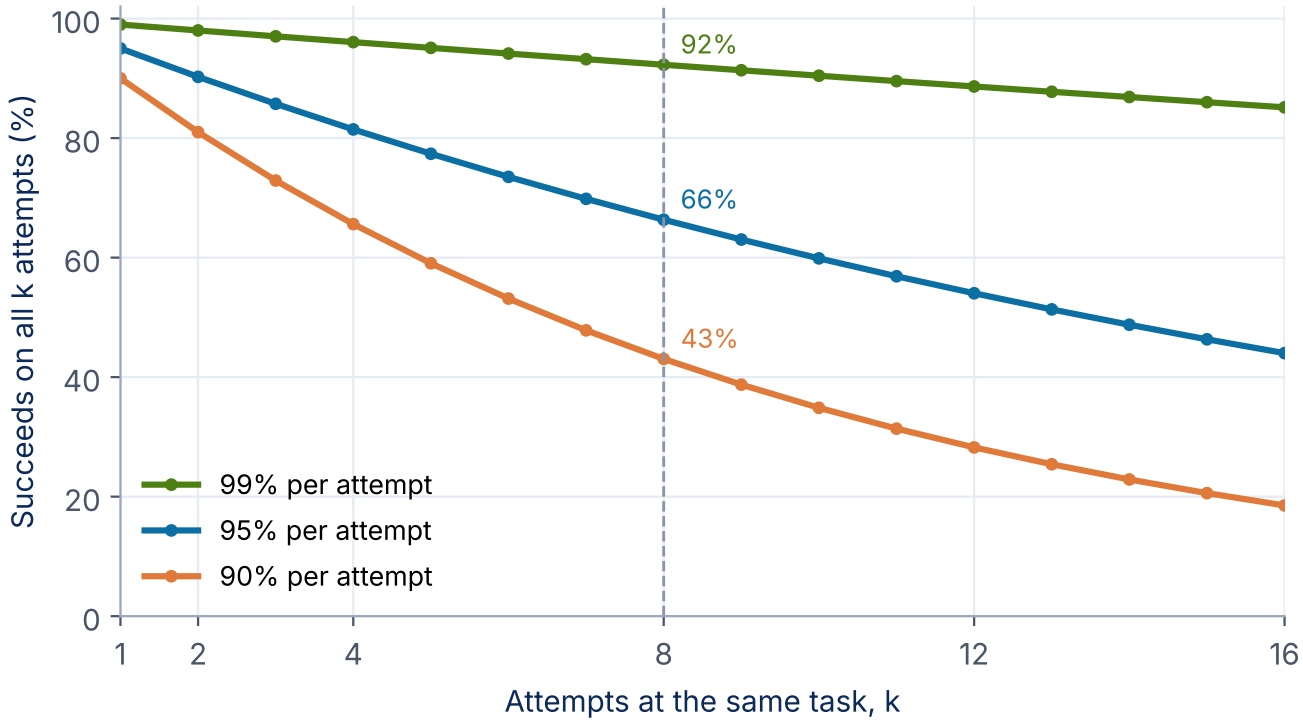


Figure 1. Probability of succeeding on all k attempts at a task, for constant per-attempt success rates, under the independence assumption (pass^k = p^k). Small differences in per-attempt success become large differences in consistency.

Author's calculation from Eq. (1) under the independence assumption.

In practice, failures cluster: some tasks are hard and fail repeatedly, others almost never fail. Eq. (1) captures that, which is why it is better than assuming p^k. The practical value of repeated runs is that they identify which tasks are unreliable, and those are often the ones that reveal a gap in the agent's instructions or data.

4.5 Size the test set for the decision

A success rate measured on a small set is uncertain. The Wilson score interval gives a confidence interval for a proportion that behaves well near 0 and 100 percent [14]. For s successes in n tasks, with $\hat{p} = s/n$ and $z = 1.96$ for 95 percent confidence:

$$p = [\ \hat{p} + z^2/2n \pm z\ \sqrt{(\ \hat{p}(1-\hat{p})/n + z^2/4n^2\ )}\ ]\ /\ (1 + z^2/n) \quad (2)$$

With 45 successes out of 50 tasks, Eq. (2) shows that the observed rate is 90 percent but that the 95 percent interval runs from 79 to 96 percent. With 360 out of 400, it narrows to 87 to 93 percent. Figure 2 shows how the width of the interval falls with the number of tasks.

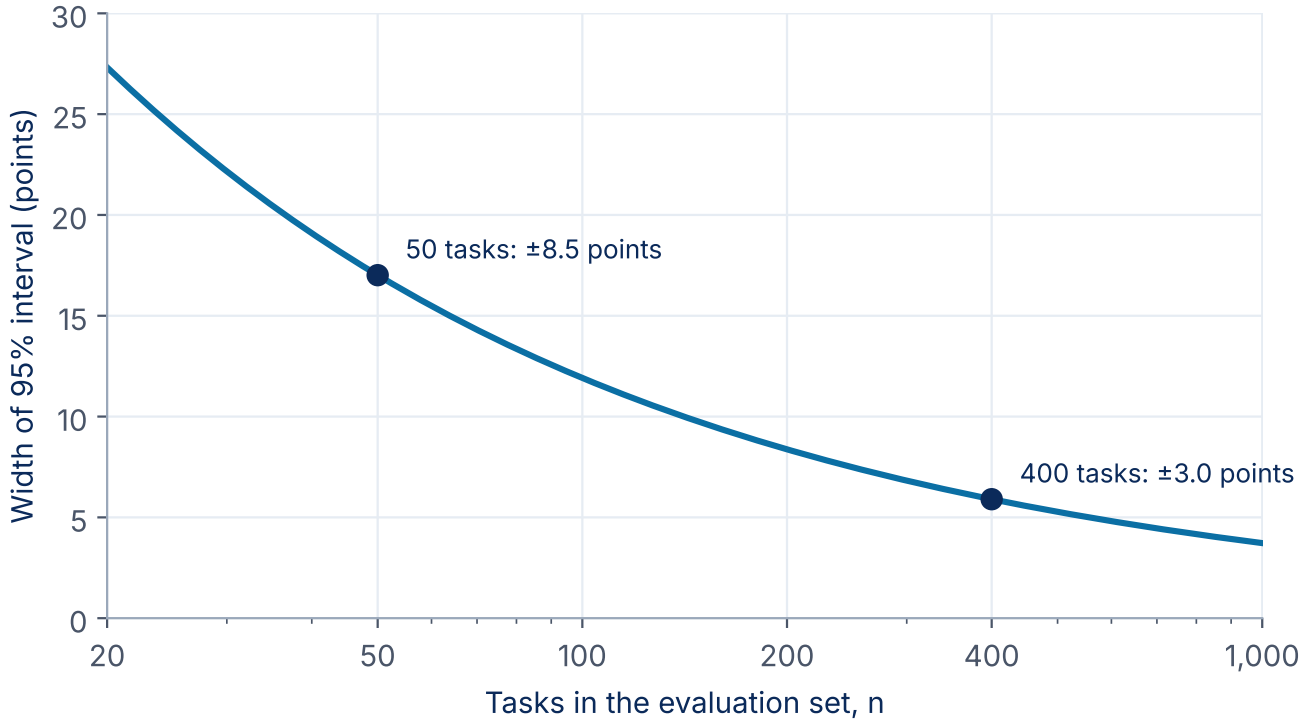


Figure 2. Width of the 95 percent Wilson confidence interval for an observed success rate of 90 percent, by the number of tasks evaluated. The interval narrows roughly with the square root of the number of tasks.

Author's calculation from Eq. (2) [14].

To plan a test set, invert the relationship. For a margin of error E around an expected success rate p, the number of tasks needed is approximately

$$n \approx z^2 \cdot p(1-p)\ /\ E^2 \quad (3)$$

To estimate a 90 percent success rate to within three percentage points at 95 percent confidence, Eq. (3) gives about 385 tasks. A set of 50 tasks is enough to tell a 60 percent agent from a 90 percent one, but not a 90 percent agent from a 95 percent one. When the question is how low the error rate is on a high-stakes action, the rule of three discussed in a companion paper gives the bound after a run of error-free trials [15].

4.6 Measure cost, time and safety alongside accuracy

Every evaluation run should record the cost and elapsed time per task, so that models and scaffolds can be compared on a cost-accuracy frontier rather than on accuracy alone [9, 10]. The set should also include safety cases: requests the agent should refuse or escalate, documents containing injected instructions, inputs outside its scope and attempts to obtain data the user is not entitled to. Table 2 summarizes the dimensions.

Table 2. Dimensions of a business evaluation for an AI agent and example metrics.

Dimension	Question	Example metrics
Correctness	Is the outcome right?	Field-level accuracy; rubric score; success rate with Wilson interval
Reliability	Is it right every time?	pass^k on repeated runs; variance by task
Policy compliance	Did it follow the rules on the way?	Unauthorized tool calls; limit breaches; missed approvals
Escalation	Does it know when to stop?	Share of out-of-scope cases escalated; false escalations
Robustness	Does it resist manipulation?	Success rate on injected-instruction cases
Cost and speed	What does it cost per task?	Tokens, tool calls, dollars and seconds per task
Drift	Is it still as good as at release?	Score on the fixed set over time; sampled production accuracy

Compiled by the author.

4.7 A worked example

Consider a business preparing to deploy an agent that codes and posts supplier invoices. It draws 400 invoices at random from the previous year and adds 60 hard cases chosen by its senior clerk: credit notes, invoices in a foreign currency, partial deliveries, a supplier whose name changed, and invoices with no matching purchase order. For each, the correct supplier, amounts, account codes, tax treatment and posting decision are recorded from the accounts. One hundred of the 460 cases are set aside as a held-out set. A further 30 safety cases are written: invoices that contain instructions to change bank details, requests to post above the agent's limit, and documents from unknown senders.

The agent runs each case three times in a test copy of the ERP system. A separate script compares the posted records with the reference values, checks the tool-call log for policy breaches and records tokens, cost and time. The first run shows 91 percent field-level accuracy, a pass^3 of 84 percent, two limit breaches and one safety case in which the agent drafted a bank-detail change. The team fixes the instructions and tool permissions, reruns the tuning set, then runs the held-out set once to confirm. The release record shows the scores with their intervals, the cost per invoice and the change made, and becomes the baseline for the next model upgrade.

5. Using a language model as the judge

For free-text outputs such as emails, summaries and investigation reports, human grading of every case is expensive, and a language-model judge is often the practical choice. It can be made reliable with four precautions drawn from the research [1, 11].

1. **Use a rubric, not an impression.** Ask the judge to score specific criteria (covers the three facts in the reference, states the correct next step, contains no promise the policy does not allow) rather than overall quality.
2. **Calibrate against people.** Have staff grade a sample of 50 to 100 cases and measure the judge's agreement with them before trusting it on the rest. Re-check after any change to the judge.
3. **Control for known biases.** Randomize the order when comparing two outputs, and do not reward length.
4. **Protect the judge.** Pass the agent's output to the judge as quoted data with clear delimiters, instruct the judge to ignore any instructions inside it, and include test cases in which the output tries to influence the grade.

Where judgment is genuinely needed, as in deciding whether an investigation report identifies the right root cause, a hybrid works well: the automated judge screens every case against the rubric, and a person reviews all cases the judge scores low, all cases where two judge runs disagree, and a random sample of the rest. This keeps human effort focused where it changes the result and gives a continuing check on the judge itself.

A judge from a different model family from the agent reduces the risk of self-enhancement bias. The judge's own prompt and model version should be fixed and recorded with each evaluation run, since a change to the judge changes the scores.

6. Using the results

Evaluation is only useful if it drives decisions. Four uses matter most.

Deployment gates. Before an agent goes live, its scores on correctness, reliability, policy compliance and safety cases should meet thresholds agreed with the business owner of the process, with intervals narrow enough to support the decision.

Autonomy decisions. The level of autonomy for each action should follow the evidence, starting from shadow-mode trials and moving up one level at a time as the error rate is shown to be low enough, with demotion on defined triggers [15].

Change control. Any change to the model, the scaffold, the tools or the instructions should be followed by a full run of the evaluation set and a comparison with the previous release. This is regression testing, and it is what allows a business to switch to a cheaper or better model with confidence as prices fall [10].

Production monitoring. The fixed set measures the agent against yesterday's work, and the business's work changes: new suppliers, new products, new policies and new kinds of customer request. A regular sample of production cases, graded the same way, measures it against today's, and catches drift in the inputs that the fixed set cannot. Cases from the sample that the agent got wrong should be added to the fixed set, so that it grows to cover the situations that actually cause trouble.

Ownership matters as much as method. The evaluation set belongs to the business owner of the process, not to the vendor or the IT team, because only the people who do the work can say what a correct outcome is. In practice this means a named owner who approves the task set and the outcome definitions, signs off release thresholds, and reviews failures with the

technical team. Vendors can supply their own test results, but those should be treated in the same way as public benchmarks: as evidence for a shortlist, to be confirmed on the business's own cases before deployment.

The capability of frontier models on long tasks has been rising quickly, with the length of task they can complete doubling roughly every seven months in METR's measurements [16]. That is a reason to re-run evaluations often, not a reason to skip them: each new model must be measured on the business's own work before it is trusted with it.

7. Conclusion

Public agent benchmarks are valuable research tools and useful for shortlisting, but their scores can be inflated by weak graders, do not transfer to a business's own tasks, depend on the scaffold, hide inconsistency and omit cost. The evidence from Berkeley and from the Agentic Benchmark Checklist shows that these are not theoretical concerns. A business can do better with modest effort: a few hundred real tasks with defined outcomes, a grader the agent cannot reach, repeated runs to measure reliability, intervals sized to the decision, and cost and safety cases alongside accuracy. Built once and rerun on every change, such an evaluation becomes the basis for deployment, autonomy and model choice, and the most durable asset in an agent program.

References

1. Wang, H., Mang, Q., Cheung, A., Sen, K., & Song, D. (2026, April). How we broke top AI agent benchmarks: And what comes next. UC Berkeley Center for Responsible, Decentralized Intelligence. rdi.berkeley.edu/blog/trustworthy-benchmarks-cont/
2. Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., Liu, S., Cui, S., Kapoor, S., Longpre, S., Meng, K., Weiss, R., et al. (2025). Establishing best practices for building rigorous agentic benchmarks. arXiv:2507.02825. arxiv.org/abs/2507.02825
3. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*. arXiv:2310.06770. arxiv.org/abs/2310.06770
4. Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., et al. (2024). WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*. arXiv:2307.13854. arxiv.org/abs/2307.13854

5. Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., et al. (2024). OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. arXiv:2404.07972. arxiv.org/abs/2404.07972
6. Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., & Scialom, T. (2023). GAIA: A benchmark for general AI assistants. arXiv:2311.12983. arxiv.org/abs/2311.12983
7. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045. arxiv.org/abs/2406.12045
8. Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., et al. (2022). Holistic evaluation of language models. arXiv:2211.09110. arxiv.org/abs/2211.09110
9. Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2024). AI agents that matter. arXiv:2407.01502. arxiv.org/abs/2407.01502
10. Lee, Q. F. (2026). The economics of agentic AI: Cost per task, total cost of ownership and payback. *Chmltech Online Journal*, 105(8). www.chmltech.com/post/the-economics-of-agentic-ai-cost-per-task-total-cost-and-p
11. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36*. arXiv:2306.05685. arxiv.org/abs/2306.05685
12. Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4902–4912). 10.18653/v1/2020.acl-main.442
13. Lee, Q. F. (2026). Agent security and identity: Prompt injection, least-privilege tool access, agent credentials and audit trails. *Chmltech Online Journal*, 105(5). www.chmltech.com/post/agent-security-and-identity-prompt-injection-least-privile
14. Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), 209–212. 10.1080/01621459.1927.10502953
15. Lee, Q. F. (2026). Levels of autonomy and human-in-the-loop design for AI agents. *Chmltech Online Journal*, 105(7). www.chmltech.com/post/levels-of-autonomy-and-human-in-the-loop-design-for-ai-age
16. Kwa, T., West, B., Becker, J., Deng, A., Garcia, K., Hasin, M., et al. (2025). Measuring AI ability to complete long tasks. arXiv:2503.14499. arxiv.org/abs/2503.14499

How to cite this article

Lee, Q. F. (2026). Evaluating AI Agents: Why Public Benchmarks Mislead and How to Build Your Own Evaluation. *Chmltech Online Journal*, *105*(10). https://www.chmltech.com/post/evaluating-ai-agents-why-benchmarks-mislead-and-how-to-build-your-own-evaluation